同濟大學

TONGJI UNIVERSITY

《数据采集与集成技术》

实验报告

实验名称	万维网运行原理分析
实验成员	刘彦（2352018）
日　　期	2025 年 10 月 13 日

1、实验目的

本实验旨在通过对特定网站的实际分析，帮助参与者深入理解万维网（www）的核心运行机制及其相关技术。实验强调从客户端浏览器到服务器端的交互过程入手，揭示网页访问背后的通信流程，包括 TCP 连接建立、HTTP 请求与响应的交换，以及网页内容的解析与展示。这种实践导向的方法，不仅能让参与者掌握万维网的结构与原理，还能培养对网络技术的直观认知。

具体而言，实验目标聚焦于三个关键方面：首先，深入剖析万维网的整体架构，包括浏览器与 Web 服务器的协作、HTTP 协议的无连接和无状态特性，以及媒体独立传输的灵活性；其次，掌握 Web 页面的组成要素，如 HTML 文档的结构、表单数据编码和多媒体资源的整合；最后，全面理解 HTTP 协议的细节，包括请求报文（请求行、头和体）的构建、响应报文的状态码与头属性（如 Cookie、Cache-Control 和 ETag）的应用，以及常见错误处理（如 404 Not Found 或 500 Internal Server Error）。

2、实验内容

2.1 网络拓扑和数据流向分析

本实验选择http://httpbin.org/forms/post作为分析网站，该网站是一个简单的示例网站，无需权限即可访问，可以发送 GET 和 POST 请求。通过浏览器开发者工具（以 Chrome 为例）观察网络过程，网络拓扑主要涉及客户端（浏览器）、传输层协议（TCP）、应用层协议（HTTP）和服务器端（Web 服务器）。

数据流向如下表所示，按照浏览器、协议、服务器、数据等维度结构化描述：

表 1 数据流向示意表

组件	描述	数据流向示例
浏览器	发起请求，解析响应，渲染页面。使用标识。	浏览器→TCP 连接→HTTP GET 请求（包含 Host: www.example.com）→服务器
协议	TCP 确保可靠传输，HTTP 处理请求/响应。无状态、无连接特性，本例为 GET 方法。	请求：方法+URL+Head（如 Accept: text/html）；响应：状态码+Head+Body
服务器	接收 TCP 请求，解析 HTTP，生成响应。常见如 Apache 或 Nginx。	服务器→TCP 响应→HTTP 200 OK（Content-Type: text/html）→浏览器
数据	流向为双向：请求携带头信息（无 Body），响应携带 HTML Body。	数据包：~1KB 请求，~500B 响应；流向闭环后关闭连接。

2.2 所选单个网页组成分析

选取http://httpbin.org/forms/post网页进行 HTML 组成要素分析。该页面结构简洁，便于对

HTML 进行分析，并符合基本 Web 页面规范。使用开发者工具查看源代码，关键要素如下：其组成要素完全遵循 HTML 的核心结构规范，可拆解为文档声明、根元素、Head 区域、Body 区域四大核心模块，各模块下包含具体功能子元素，详细分析如下：

①文档声明（Document Type Declaration）

这是 HTML5 的标准文档声明，用于告诉浏览器“当前文档遵循 HTML5 语法规则”，浏览器会据此以 HTML5 模式解析页面内容，避免进入兼容旧版本的 Quirks Mode，确保表单元素（如 type=tel、type=time）等 HTML5 新特性正常渲染。

②根元素（Root Element）

是整个 HTML 文档的“根容器”，所有其他 HTML 元素（<head>、<body>）都必须嵌套在<html>标签内部，是文档结构的顶层节点，浏览器通过它识别文档的 HTML 类型。

③头部区域（Head Section）

用于定义网页的“元数据”（不直接显示在页面上的信息），如字符编码、页面标题、外部资源（CSS、JavaScript）引用等。

④主体区域（Body Section）

包含所有 “用户可见的内容”，如文本、图片、表单、按钮等，是网页的核心展示区域。本页面的主体内容是一个完整的披萨订单表单，内部按功能拆解为以下子元素：

● 表单容器（Form Element）

包裹所有表单控件（输入框、单选框、复选框等），是表单功能的 “总容器”。

● 表单控件组（Form Controls）

表单内部的控件按“功能类型”分类，均通过<label>标签与控件关联（点击标签可激活对应控件，提升用户体验）：

● 段落容器（Paragraph Element）

将每个表单控件（含<label>）包裹在<p>标签中，通过默认的段落间距（margin）实现控件之间的垂直分隔，避免控件拥挤，提升表单的视觉层次感和可读性。

2.3 所选网页的 HTTP 协议分析

通过浏览器开发者工具捕获对http://httpbin.org/forms/post的 GET 请求和 POST 请求过程，HTTP 协议基于 TCP/IP 的无连接、无状态和媒体独立模型，支持 GET（查询资源）和 POST（提交数据）等方法，确保高效传输超文本和表单数据。

GET 请求报文概括：采用 GET 方法，请求行指定 URL（如/）和 HTTP/1.1 版本；头属性包括 Accept（支持 MIME 类型如 text/html）、Host（域名标识）和 User-Agent（浏览器信息）；体部为空，参数可选嵌入 URL（如?param=value）。关键点在于无状态下需重传信息，Referer 追踪来源，适用于资源获取场景。

POST 请求报文概括：采用 POST 方法，请求行指定 URL（如/post）和 HTTP/1.1 版本；头属性包括 Content-Type（application/x-www-form-urlencoded，表单编码）和 Accept；体部携带键

值对数据（如 username=tom&password=123），参数隐藏于 Body。关键点在于提升安全性，Cookie 维护会话（如 JSESSIONID），适用于数据提交场景。

响应报文概括：两者均返回 200 OK 状态行；头属性涵盖 Content-Type（text/html 或 application/json）、Cache-Control（缓存策略，如 max-age=3600）和 Date（时间戳）；体部为 HTML 或 JSON 内容，镜像请求数据。状态码分类（1xx 接收、2xx 成功、3xx 重定向、4xx/5xx 错误）提供反馈，如 304 Not Modified 优化缓存，303 See Other 支持重定向。

整体分析验证 HTTP 的 C/S 架构：GET 适合读取、POST 适合修改，头属性如 ETag 版本控制和 Set-Cookie 会话管理增强交互。多次操作确认协议可靠，持久连接优化多请求传输。

3、实验步骤

3.1 网络拓扑和数据流向分析

现代 Web 请求分析的核心在于将网络拓扑和数据流向结构化地分解为四个关键要素：浏览器、协议、服务器和数据。浏览器作为起点，负责发起 DNS 查询并构造 HTTP/HTTPS 请求，定义了请求的客户端环境。请求随后进入协议层，在这里完成 TCP/IP 连接的建立、路由选择，以及通过 TLS/SSL 进行加密，确定了数据传输的路径和安全机制。当请求到达服务器端，它依次经过负载均衡、Web/应用服务器进行业务逻辑处理，并可能查询数据库，这定义了数据处理的物理和逻辑拓扑结构。最后，数据 作为整个流程的最终产物，无论是动态生成的 JSON/HTML 响应还是静态资源，都会沿着协议层逆向返回给浏览器，并同时流入日志、缓存和持久化存储系统，构成了数据的最终流向和利用。

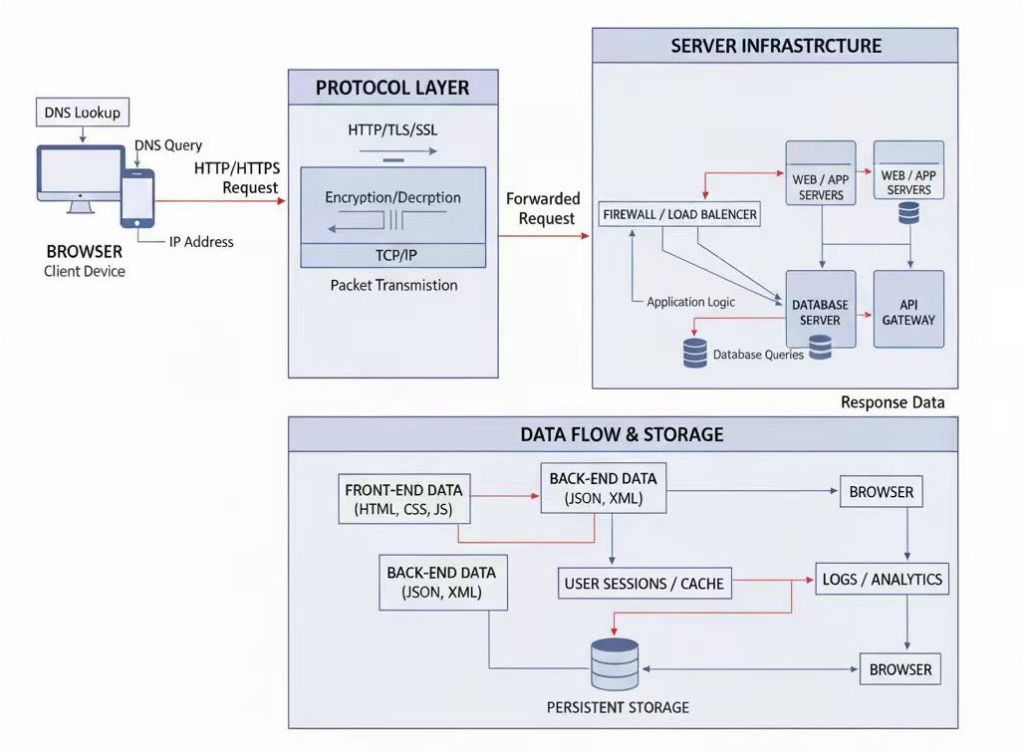


图 1 网络拓扑和数据流向示意图

3.2 所选单个网页组成分析

①文档类型声明（Document Type Declaration）

<!DOCTYPE html>：这行代码位于最顶端，它向浏览器声明了该文档是一个 HTML5 文档。这不是一个 HTML 标签，而是一个指令，确保浏览器以“标准模式”来渲染页面。

②根元素（Root Element）

<html>：这是整个 HTML 文档的根元素，所有其他元素都嵌套在它内部。

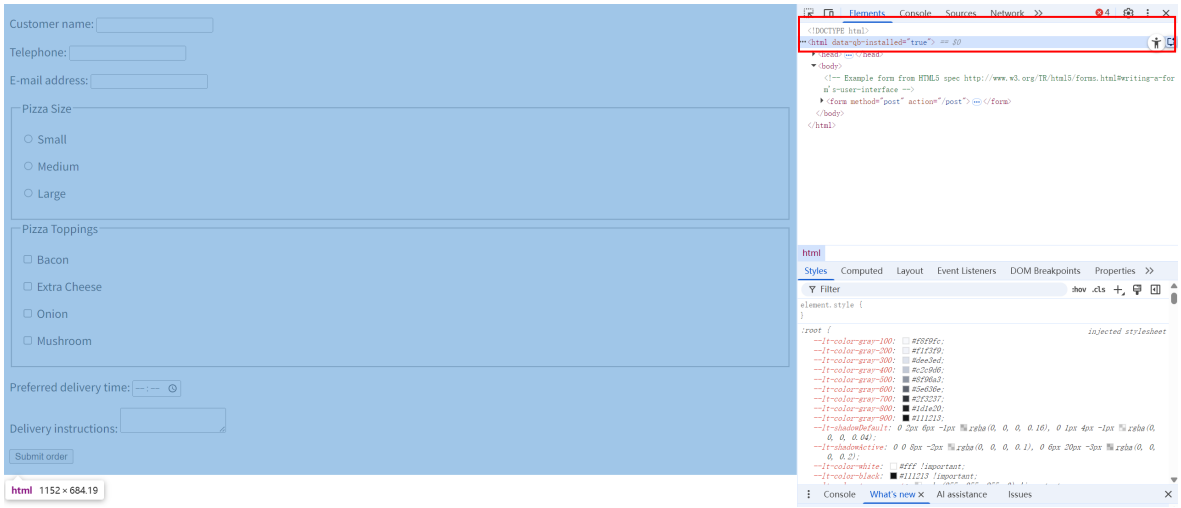


图 2 该网页 HTML 文档的根元素及其 CSS 规则

③头部元素（Head Element）

<head>：这个部分包含了文档的元数据（meta-data），也就是描述文档自身信息的内容，这些信息通常不会直接显示在页面上给用户看。

在此示例中，<head>标签是空的（只有 style），但通常它会包含诸如<title>（页面标题）、<meta>(字符集设置等)、<link>(链接 CSS 样式表)和<script>（链接 JavaScript 文件）等标签。

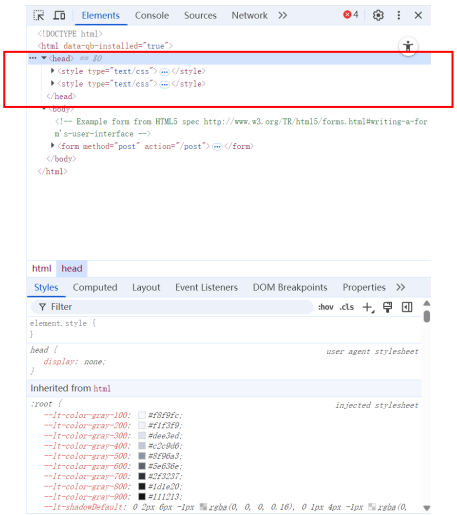


图 3 该网页 HTML 文档的头部元素及其 CSS 规则

例如在 github.com 中，<head>就包含上面说叙述的若干标签。

```
▼ <head> == $0
  ▶ <style type="text/css">…</style>
    <meta charset="utf-8">
    <link rel="dns-prefetch" href="https://github.githubassets.com">
    <link rel="dns-prefetch" href="https://avatars.githubusercontent.com">
    <link rel="dns-prefetch" href="https://github-cloud.s3.amazonaws.com">
    <link rel="dns-prefetch" href="https://user-images.githubusercontent.c
    om/">
    <link rel="preconnect" href="https://github.githubassets.com"
    crossorigin>
    <link rel="preconnect" href="https://avatars.githubusercontent.com">
    ⋮
  ▶ <script type="application/json" id="client-env">…</script>
    <script crossorigin="anonymous" type="application/javascript" src="htt
    ps://github.githubassets.com/assets/high-contrast-cookie-f3788027bd8d.
    js"></script>
    <script crossorigin="anonymous" type="application/javascript" src="htt
    ps://github.githubassets.com/assets/wp-runtime-481d502ec491.js" defer=
    "defer"></script>
    ⋮
```

图 4 github 网页 HTML 文档的头部元素

④主体元素（Body Element）

<body>：这个部分包含了构成网页可见内容的所有元素，例如文本、图片、表单、链接等。用户在浏览器窗口中看到的所有内容都位于<body>标签之内。

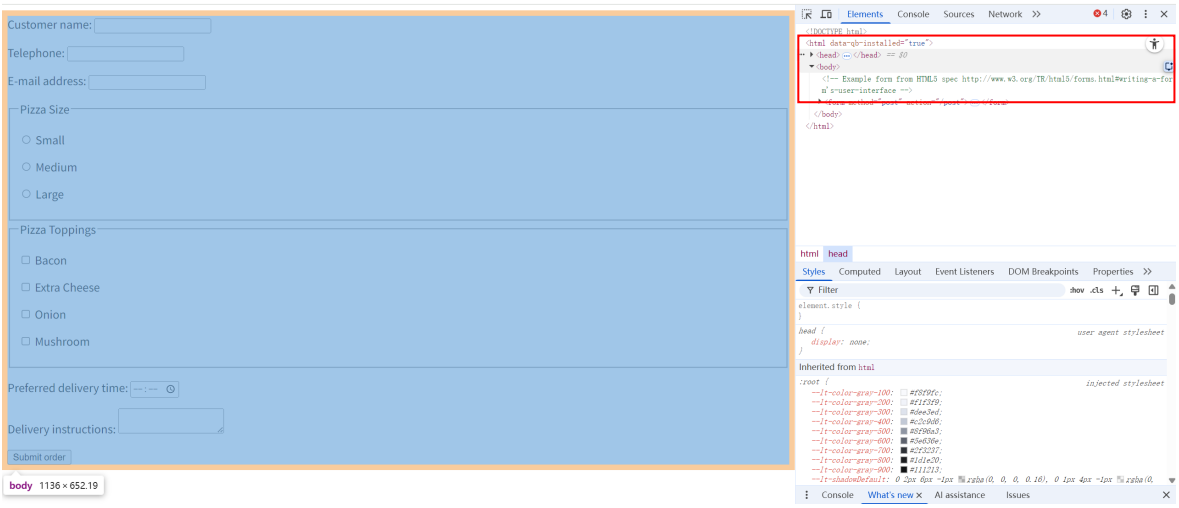


图 5 该网页 HTML 文档的主体元素及其 CSS 规则

⑤表单元素（Form Element）

<form method="post" action="/post">: 这是页面的核心功能元素，用于收集用户信息。method="post"这个属性指定了在提交表单时使用的 HTTP 方法。POST 方法会将表单数据包含在 HTTP 请求的主体中发送到服务器，通常用于提交敏感信息或大量数据。action="/post"这个属性定义了当用户提交表单后，表单数据应该被发送到哪个 URL 进行处理。在这里，数据将被发送到当前域名下的/post 路径。

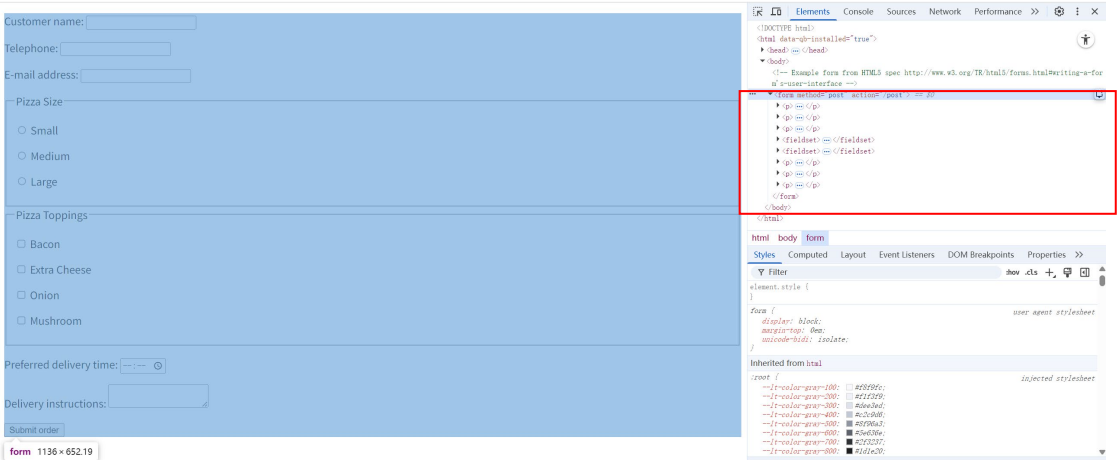


图 6 该网页 HTML 文档的表单元素及其 CSS 规则

⑥表单控件和相关元素（Form Controls and Associated Elements）

该页面主体内容完全由一个表单构成，表单内部包含了多种用于数据输入的控件，并通过段落和标签来组织结构。

段落**<p>**：用于组织和分隔表单中的不同输入项，使布局更加清晰。

标签**<label>**：为表单控件提供描述性文本。它通过将文本和控件（如<input>）包裹起来，提升了页面的可用性。例如，用户可以点击标签文本来聚焦或选中对应的输入框或选项。

文本输入框**<input>**

- <input name="custname">：一个基础的单行文本输入框，用于输入顾客姓名。name 属性"custname"是提交到服务器时该字段的唯一标识。
- <input type="tel" name="custtel">：一个专门用于输入电话号码的输入框。在移动设备上，type="tel" 会优化屏幕键盘，方便数字输入。
- <input type="email" name="custemail">：一个专门用于输入电子邮件地址的输入框。浏览器可以利用这个类型来验证输入格式是否符合标准的 Email 格式。

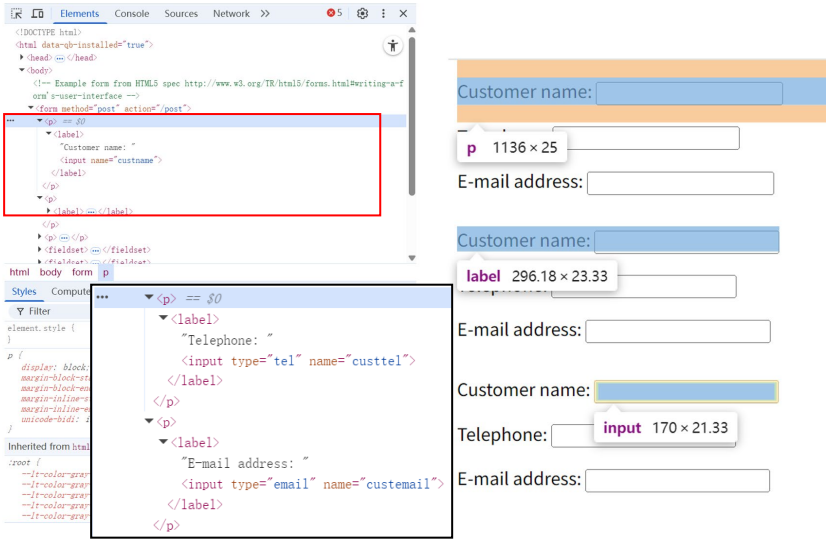


图 7 该网页 HTML 文档的段落、标签和文本输入框元素及其 CSS 规则

- <input type="radio">：用于在一组选项中只选择一个。所有相关的单选按钮（例如“Small”、“Medium”、“Large”）拥有相同的 name="size" 属性，这确保了用户一次只能选择一个尺寸。value 属性（如"small"）定义了当该选项被选中并提交时，发送到服务器的值。

```
<p>
  <label>
    <input type="radio" name="size" value="small">
    " Small "
  </label>
</p>
<p>
  <label>
    <input type="radio" name="size" value="medium">
    " Medium "
  </label>
</p>
<p>
  <label>
    <input type="radio" name="size" value="large">
    " Large "
  </label>
</p>
```

图 8 该网页 HTML 文档的选择框元素及其 CSS 规则

- <input type="checkbox">：用于从一组选项中进行多项选择。每个复选框都有相同的 name="topping"，当表单提交时，所有被选中的 value 都会被发送。
- <input type="time">：提供一个用于选择时间的控件。min="11:00"和 max="21:00"属性限制了可选的时间范围（从上午 11 点到晚上 9 点）。step="900"属性定义了时间的步长，单位是秒，所以时间选项会以 15 分钟为间隔。

```
<p>
  <label>
    <input type="checkbox" name="topping" value="bacon">
    " Bacon "
  </label>
</p>
<p>
  <label>
    <input type="checkbox" name="topping" value="cheese">
    " Extra Cheese "
  </label>
</p>
<p>
  <label>
    <input type="checkbox" name="topping" value="onion">
    " Onion "
  </label>
</p>
<p>
  <label>
    <input type="checkbox" name="topping" value="mushroom">
    " Mushroom "
  </label>
</p>
</fieldset>
<p>
  <label>
    "Preferred delivery time: "
    <input type="time" min="11:00" max="21:00" step="900" name="delivery">
  </label>
```

图 9 该网页 HTML 文档的复选框和时间输入元素及其 CSS 规则

字段集**<fieldset>**和图例**<legend>**：

- <fieldset>：用于将表单内的相关控件组合在一起，并在视觉上用边框将它们包围起来，增强了表单的结构性和可读性。
- <legend>：为<fieldset>提供一个标题或说明。例如，“Pizza Size”和“Pizza Toppings”清

晰地说明了每个字段集的目的。

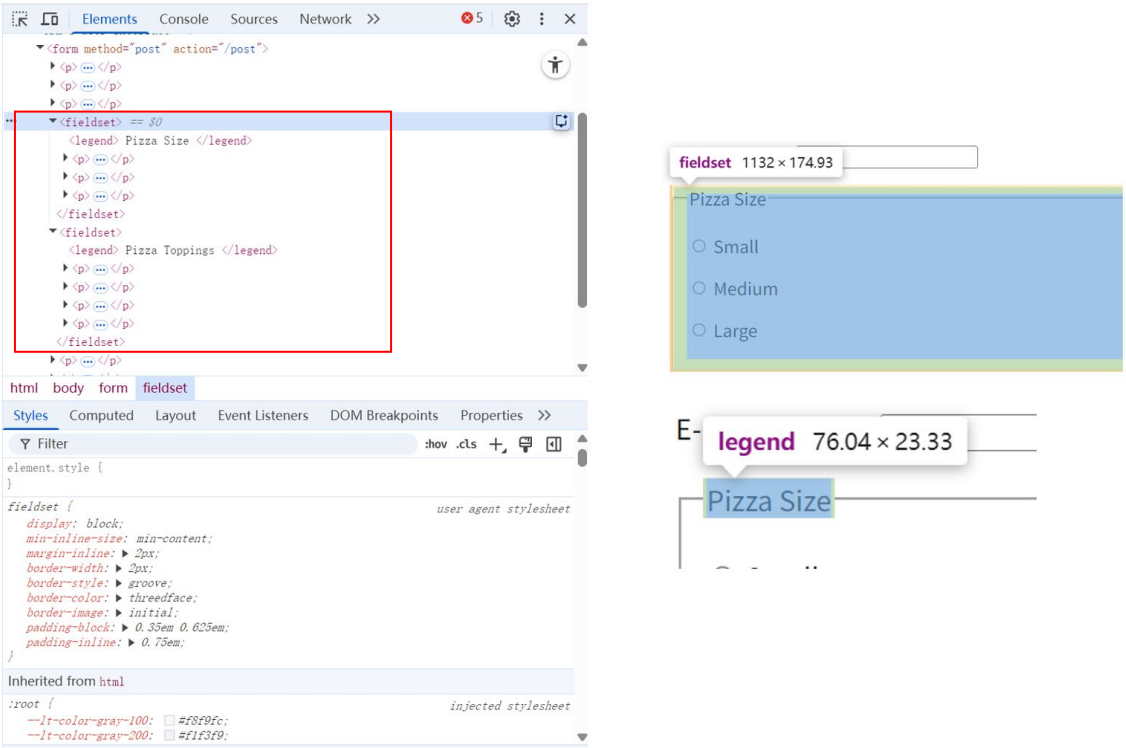


图 10 该网页 HTML 文档的字符集和图例元素及其 CSS 规则

多行文本域**<textarea>**：<textarea name="comments"></textarea>提供一个多行文本输入区域，用于用户输入较长的文本，如配送说明。

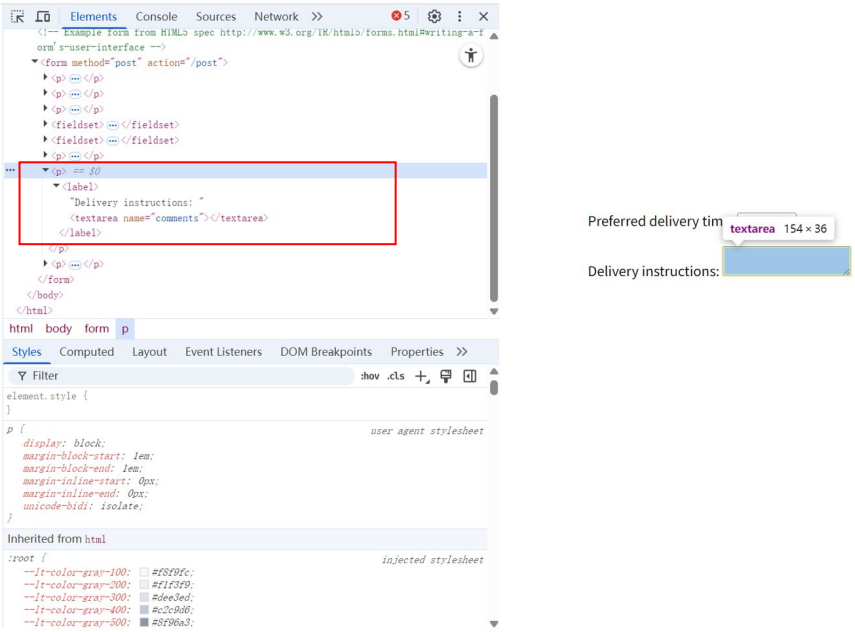


图 11 该网页 HTML 文档的多行文本域元素及其 CSS 规则

提交按钮**<button>**：<button>Submit order</button>是一个按钮，当用户点击它时，会触发表

单的提交事件，将所有收集到的数据发送到 action 属性指定的地址。

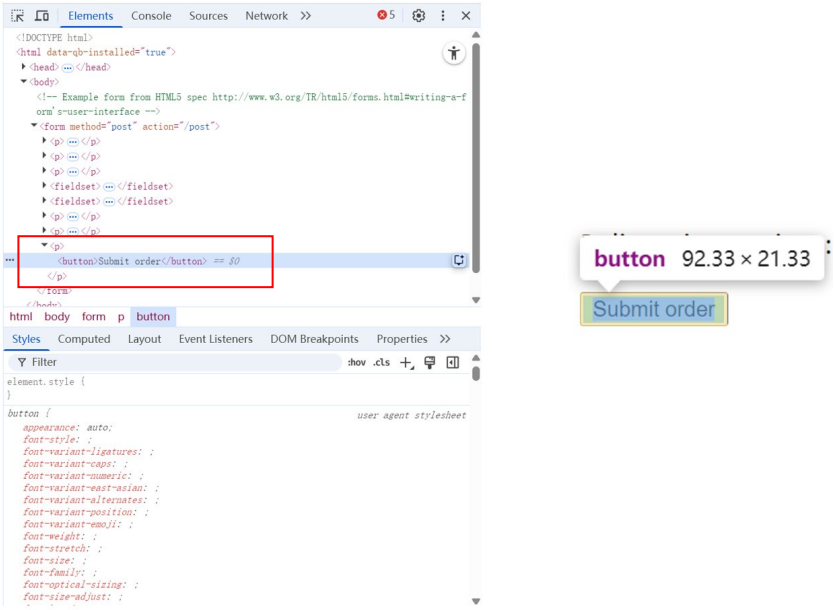


图 12 该网页 HTML 文档的提交按钮元素及其 CSS 规则

3.3 所选网页的 HTTP 协议分析

①GET

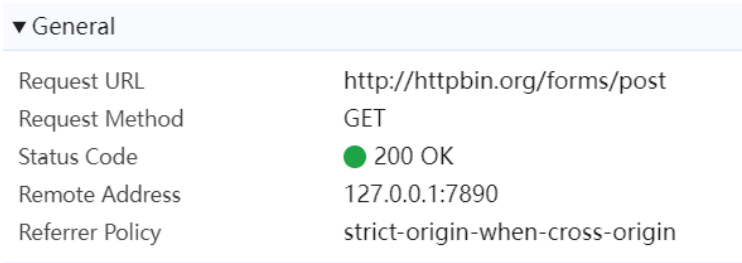


图 13 该网页 GET 请求通用设置

打开网页，获得 GET 请求报文：

```
GET /forms/post HTTP/1.1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9
Host: httpbin.org
Proxy-Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
```

▾ Request Headers	☐ Raw
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate
Accept-Language	zh-CN,zh;q=0.9
Host	httpbin.org
Proxy-Connection	keep-alive
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36

图 14 该网页 GET 请求报文要素

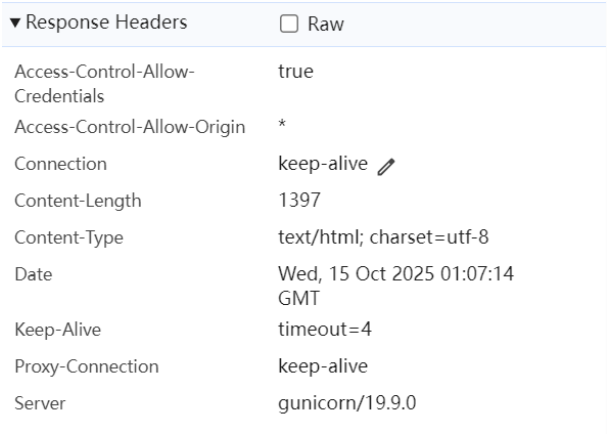
该请求报文是运行在 Windows 64 位系统上的 Chrome 141.0.0.0 浏览器，依据 HTTP/1.1 协议向目标服务器 httpbin.org 发起的 GET 类型请求，核心目的是获取服务器上/forms/post 路径对应的资源，且未附带请求体数据。

从请求头部细节来看，浏览器明确告知服务器：优先接收 text/html（HTML 页面）、image/avif 等格式的资源，同时支持 gzip 和 deflate 两种数据压缩算法以减少传输流量；语言偏好上，优先选择中文（中国大陆，zh-CN），其次兼容其他中文变体（zh）；网络连接方面，通过代理与服务器保持长连接（keep-alive）以提升后续请求效率，并主动请求将 HTTP 协议升级为 HTTPS 以增强传输安全性。整体符合标准的 HTTP 资源获取请求结构，各项头部信息完整且明确，可帮助服务器精准返回适配客户端环境的响应内容。

获得 GET 应答报文：

```
HTTP/1.1 200 OK
Content-Length: 1397
Access-Control-Allow-Credentials: true
Access-Control-Allow-Origin: *
Connection: keep-alive
Content-Type: text/html; charset=utf-8
Date: Wed, 15 Oct 2025 01:07:14 GMT
Keep-Alive: timeout=4
Proxy-Connection: keep-alive
Server: gunicorn/19.9.0
```

装 订 线



▾ Response Headers ☐ Raw

Access-Control-Allow-Credentials	true
Access-Control-Allow-Origin	*
Connection	keep-alive
Content-Length	1397
Content-Type	text/html; charset=utf-8
Date	Wed, 15 Oct 2025 01:07:14 GMT
Keep-Alive	timeout=4
Proxy-Connection	keep-alive
Server	gunicorn/19.9.0

图 15 该网页 GET 应答报文要素

该响应报文是服务器（使用 Gunicorn/19.9.0 软件）针对之前的 GET 请求，依据 HTTP/1.1 协议返回的成功响应，状态码 200 OK 表明请求已被正常处理。响应将向客户端返回大小为 1397 字节、格式为 text/html 且字符编码为 UTF-8 的 HTML 页面资源，确保浏览器能正确解析且不出现乱码；响应时间标注为 2025 年 10 月 15 日 01:07:14 GMT，方便追溯处理节点。同时，服务器支持跨域访问，允许所有域名（Access-Control-Allow-Origin: *）发起跨域请求，且允许客户端携带身份凭证（如 Cookie）；在连接策略上，服务器同意与客户端通过代理保持长连接（Connection 和 Proxy-Connection 均为 keep-alive），并设置超时时间为 4 秒，若 4 秒内无新请求则关闭连接，既提升后续请求效率，又避免资源浪费。

②POST

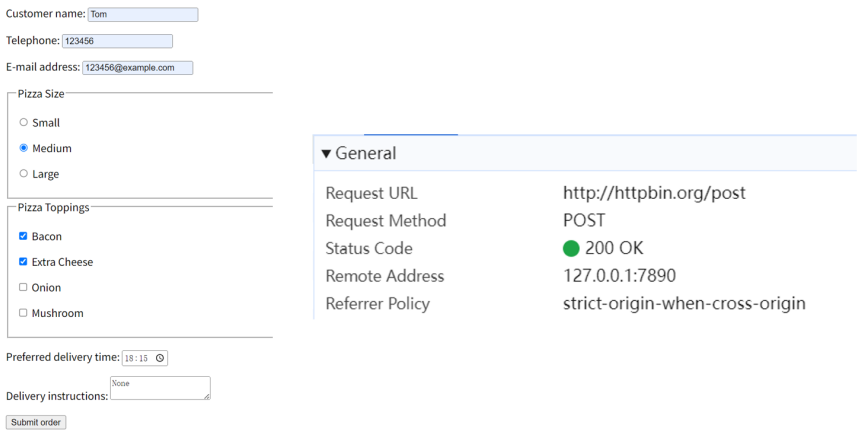


图 16 该网页 POST 请求通用设置和提交信息

提交信息，获得 POST 请求报文：

```
POST /post HTTP/1.1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9
Cache-Control: max-age=0
Content-Length: 130
Content-Type: application/x-www-form-urlencoded
Host: httpbin.org
Origin: http://httpbin.org
Proxy-Connection: keep-alive
Referer: http://httpbin.org/forms/post
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
```

▾ Request Headers	☐ Raw
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate
Accept-Language	zh-CN,zh;q=0.9
Cache-Control	max-age=0
Content-Length	130
Content-Type	application/x-www-form-urlencoded
Host	httpbin.org
Origin	http://httpbin.org
Proxy-Connection	keep-alive
Referer	http://httpbin.org/forms/post
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36

图 17 该网页 POST 请求报文要素

该请求报文是运行在 Windows 64 位系统上的 Chrome 141.0.0.0 浏览器，通过 HTTP/1.1 协议向 httpbin.org 服务器的/post 路径发起发起的 POST 请求。请求将提交 130 字节的数据，内容类型为 application/x-www-form-urlencoded（表单默认编码格式），表明客户端正在提交表单数据。报文头部显示，浏览器优先接收 HTML、XML 等格式内容，支持 gzip 和 deflate 压缩，偏好中文语言；同时设置 Cache-Control: max-age=0 表示不使用缓存需获取最新资源，Origin 和 Referer 字段分别指明请求来源域为 http://httpbin.org 及具体页面/forms/post，说明这是从该网站内部表单页面发起的提交。此外，请求通过代理保持长连接，并希望升级为 HTTPS 以提升安全性，整体符合标准的表单数据提交请求结构，各项头部信息完整，可帮助服务器正确接收和处理提交的数据。

获得 POST 应答报文：

装 订 线

```
HTTP/1.1 200 OK
Content-Length: 1104
Access-Control-Allow-Credentials: true
Access-Control-Allow-Origin: http://httpbin.org
Connection: keep-alive
Content-Type: application/json
Date: Wed, 15 Oct 2025 04:22:40 GMT
Keep-Alive: timeout=4
Proxy-Connection: keep-alive
Server: gunicorn/19.9.0
```

▼ Response Headers ☐ Raw

Access-Control-Allow-Credentials	true
Access-Control-Allow-Origin	http://httpbin.org
Connection	keep-alive
Content-Length	1104
Content-Type	application/json
Date	Wed, 15 Oct 2025 04:22:40 GMT
Keep-Alive	timeout=4
Proxy-Connection	keep-alive
Server	gunicorn/19.9.0

图 18 该网页 POST 应答报文要素

该响应报文是使用 Gunicorn/19.9.0 服务器软件，依据 HTTP/1.1 协议，针对之前的 POST 请求返回的成功响应，状态码 200 OK 表明客户端提交的数据已被服务器正常接收并处理。响应将向客户端返回大小为 1104 字节、格式为 application/json 的 JSON 数据，确保客户端能按 JSON 格式解析处理返回结果；响应生成时间标注为 2025 年 10 月 15 日 04:22:40 GMT，可用于追溯请求处理的时间节点。在跨域配置上，服务器仅允许来源为 http://httpbin.org 的请求进行跨域访问（而非通配符 *），同时支持客户端携带身份凭证（如 Cookie），符合同源请求的安全策略；连接方面，服务器同意与客户端通过代理保持长连接（Connection 和 Proxy-Connection 均为 keep-alive），并设置 4 秒超时时间，若 4 秒内无新请求则关闭连接，既保障后续请求的通信效率，又避免服务器资源被长期占用。

返回 JSON 数据如下：


```
{
  "args": {

  },
  "data": "",
  "files": {

  },
  "form": {
    "comments": "None",
    "customail": "1234560@example.com",
    "custname": "Tom",
    "custtel": "123456",
    "delivery": "18:15",
    "size": "medium",
    "topping": [
      "bacon",
      "cheese"
    ]
  },
  "headers": {
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7",
    "Accept-Encoding": "gzip, deflate",
    "Accept-Language": "zh-CN,zh;q=0.9",
    "Cache-Control": "max-age=0",
    "Content-Length": "130",
    "Content-Type": "application/x-www-form-urlencoded",
    "Host": "httpbin.org",
    "Origin": "http://httpbin.org",
    "Referer": "http://httpbin.org/forms/post",
    "Upgrade-Insecure-Requests": "1",
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36",
    "X-Amzn-Trace-Id": "Root=1-68ef2190-2cd16b712f459bce2721ec84"
  },
  "json": null,
  "origin": "111.187.124.224",
  "url": "http://httpbin.org/post"
}
```

4、实验结果

4.1 网络拓扑和数据流向分析结果

通过对http://httpbin.org/forms/post示例网站的分析，网络拓扑主要由客户端浏览器、TCP 传输层、HTTP 应用层和 Web 服务器组成，形成客户端-服务器（C/S）架构。数据流向呈双向闭环：浏览器发起 TCP 连接并发送 HTTP GET 请求（包含 Host 头和 Accept 头，无 Body，约 1KB），服务器接收后返回 HTTP 200 OK 响应（包含 Content-Type: text/html 和 Body，约 500B），连接随后关闭。TCP 确保可靠传输，HTTP 体现无连接、无状态特性，支持媒体独立传输。该流程验证了从请求发起到响应渲染的完整交互，强调长连接（keep-alive）优化多请求效率（图 1）。

4.2 所选单个网页组成分析结果

选取 http://httpbin.org/forms/post 页面，其 HTML 结构严格遵循 HTML5 规范，通过实验了解其核心模块包括：①文档声明（），确保标准模式解析；②根元素（<html>），作为顶层容器（图 2）；③头部区域（<head>），定义元数据（图 3、4）；④主体区域（<body>），包含用户可见内容，主要为披萨订单表单。表单要素详析（图 5）：⑤表单容器，指定 POST 提交路径（图 6）；⑥控件组，包括关联的（文本如 custname、tel、email；单选 radio 如 size；复选 checkbox 如 topping；时间 time 限制 11:00-21:00），段落分隔布局（图 7、8、9）；<fieldset>和<legend>分组（图 10）；<textarea>多行输入（图 11）；<button>提交（图 12）。整体结构提升可用性和视觉层次，符合 Web 页面规范。

4.3 所选网页的 HTTP 协议分析结果

通过浏览器开发者工具（Chrome）捕获并分析 http://httpbin.org/forms/post 页面的 GET 和 POST 请求/响应过程，验证 HTTP/1.1 协议的无连接、无状态和媒体独立特性，以及 C/S 架构下的交互机制。

具体结果包括：打开页面发起 GET 请求（图 13），观察请求报文（GET /forms/post HTTP/1.1，无 Body）和头域（Accept 优先 text/html、Host: httpbin.org、User-Agent: Chrome 141、Connection: keep-alive），以上见图 14，记录应答报文（200 OK，Gunicorn/19.9.0 服务器，头含 Content-Type: text/html; charset=utf-8、Date: 2025-10-15 GMT、Access-Control-Allow-Origin: * 和 keep-alive 4s 超时，Body 1397B HTML），以上见图 15。

随后填写表单提交 POST 请求（图 16），捕获请求报文（POST /post HTTP/1.1，Body 130B application/x-www-form-urlencoded 数据，如 custname=tom）和头域（Content-Type、Origin/Referer: http://httpbin.org/forms/post、Cache-Control: max-age=0、Upgrade-Insecure-Requests: 1），以上见图 17，记录应答报文（200 OK，头含 Content-Type: application/json、Access-Control-Allow-Origin: http://httpbin.org 和 keep-alive，Body 1104B JSON 镜像数据），以上见图 18。

通过这些步骤，总结验证了 GET 用于资源获取（Referer 追踪来源）、POST 用于数据提交（Body 隐藏参数提升安全），头域如 ETag/Cookie 优化缓存与会话，状态码分类（2xx 成功等）提供反馈，

持久连接提升效率，整体确认协议可靠性和表单闭环交互。

5、实验总结

通过本次实验，我们对 http://httpbin.org/forms/post 示例网站的网络拓扑、HTML 网页组成以及 HTTP 协议交互进行了系统分析，达到了预期的学习目标，即深入理解万维网的核心运行机制及其技术原理。实验过程从客户端浏览器发起请求入手，逐步剖析了 TCP 连接建立、HTTP 请求/响应交换，以及网页内容的解析与渲染，验证了 Web 系统的 C/S 架构、无连接无状态特性，以及媒体独立传输的灵活性。同时，通过开发者工具的实际捕获和拆解，我们掌握了 HTML5 文档的结构规范（从 DOCTYPE 到 Body 表单控件）和 HTTP 报文的构建细节（GET/POST 方法、头域如 Accept、Content-Type、Cache-Control 的应用，以及状态码如 200 OK 的反馈机制），这不仅加深了对 Web 技术的理论认知，还培养了实践调试能力。

在问题处理方面，实验虽基于 http://httpbin.org/forms/post 这样一个简洁的示例网页（仅一个表单，无复杂多媒体或动态脚本），但仍让我遇到了几处常见挑战，锻炼了调试技巧。首先，开发者工具的网络面板在模拟高延迟环境下偶尔卡顿，导致捕获的报文时间戳（如 Date 头）需多次刷新验证。其次，特殊字符输入（如中文姓名在 custname 字段）若未 URL 编码，可能在 Body 中解析出错，返回 500 Internal Server Error，这让我强调了在开发中统一采用 UTF-8 并测试边缘案例。这些问题虽因网页简单而易于定位，却让我养成“预防为主”的习惯，快速迭代解决方案。

剖析 http://httpbin.org/forms/post 表单时，我发现关联和<fieldset>分组并非简单装饰，而是“用户为中心”的体现——type="tel"输入框对移动端的适配，极大提升了包容性设计。这让我反思：在未来的前端开发中，应优先考虑无障碍访问和响应式布局，避免追求美观而忽略人性化。同时，实验激发了我对 Web 演进的兴趣：从 HTTP/1.1 的无连接到 HTTP/3 的 QUIC 优化，从静态 HTML 到动态 React 生态，每一步迭代都源于协作与需求驱动。这鼓励我进一步探索 HTTPS 加密、WebSocket 实时通信，甚至 API 设计，以构建更 robust 的网络应用。

装 订 线